

Exos Complémentaires

1. (RS) On suppose que eax, ebx et ecx contiennent trois valeurs (des entiers signés).
Écrire une suite d'instructions qui place dans edx la plus grande de ces trois valeurs.

```
mov edx, eax      ; edx = eax (on suppose que eax est le max pour l'instant)
```

```
cmp ebx, edx      ; compare ebx avec edx (le max actuel)
jle suite1        ; si ebx <= edx, on saute
mov edx, ebx      ; sinon, ebx est plus grand, donc edx = ebx
```

```
suite1:
cmp ecx, edx      ; compare ecx avec edx (le max actuel)
jle fin           ; si ecx <= edx, on a fini
mov edx, ecx      ; sinon, ecx est plus grand, donc edx = ecx
```

```
fin:
```

2. (RS) Écrire une suite d'instructions qui, dans le cas où eax et ebx contiennent des valeurs d'un même signe, remplace le contenu de ecx par son opposé.

```
xor eax, ebx      ; xor des deux nombres
sar eax, 31        ; décale le bit de signe à la position 0 (eax = 0 ou -1)
jne .fin          ; si eax != 0 (signes différents) -> ne rien faire
```

```
neg ecx           ; sinon, même signe -> ecx = -ecx
```

```
.fin:
```

Avant sar eax, 31 :

eax = 1xxxxx...xxxxx (bit de signe = 1)

ou

eax = 0xxxxx...xxxxx (bit de signe = 0)

Après sar eax, 31 :

eax = 0xFFFFFFFF (tous les bits à 1, = -1 en signé)

ou

eax = 0x00000000 (tous les bits à 0, = 0)

Concrètement :

Si le nombre était positif ou nul → bit de signe = 0 → sar remplit avec des 0 → résultat = 0

Si le nombre était négatif → bit de signe = 1 → sar remplit avec des 1 → résultat = 0xFFFFFFFF = -1

3. (RSB) On suppose que `eax` contient une valeur positive que l'on symbolise par n . Écrire une suite d'instructions faisant en sorte de placer la valeur $1 + 2 + 3 + \dots + n$ dans `eax`.

```
mov edx, eax    ; edx = n
mov ebx, 1      ; ebx = 1 (compteur)
mov eax, 0      ; eax = 0 (somme)

boucle:
  add eax, ebx  ; eax = eax + ebx (accumule)
  inc ebx      ; ebx++ (prochain nombre)
  cmp ebx, edx  ; compare ebx avec n
  jle boucle    ; si ebx <= n, on reboucle
```

; fin : `eax` contient $1+2+3+\dots+n$

4. (RSB) On suppose que `eax` contient une valeur positive que l'on symbolise par n . Écrire une suite d'instructions faisant en sorte de placer la valeur $1^2 + 2^2 + 3^2 + \dots + n^2$ dans `eax`.

```
mov edx, eax    ; edx = n
mov ebx, 1      ; ebx = 1 (compteur)
mov eax, 0      ; eax = 0 (somme des carrés)

boucle:
  mov ecx, ebx  ; ecx = ebx
  imul ecx, ebx ; ecx = ecx * ebx (ecx = ebx²)
  add eax, ecx  ; eax = eax + ecx (accumule le carré)
  inc ebx      ; ebx++ (prochain nombre)
  cmp ebx, edx  ; compare ebx avec n
  jle boucle    ; si ebx <= n, on reboucle
```

; fin : `eax` contient $1^2+2^2+3^2+\dots+n^2$

5. (RP) Écrire une suite d'instructions qui échange le contenu des valeurs des registres `eax` et `ebx`. Les valeurs des deux autres registres de travail `ecx` et `edx` doivent être préservées (indice : utiliser la pile).

```
push eax        ; empile eax sur la pile
push ebx        ; empile ebx sur la pile

pop ebx         ; dépile dans ebx (récupère ancienne valeur de eax)
pop eax         ; dépile dans eax (récupère ancienne valeur de ebx)
```

6. (RP) On suppose que la pile contient trois données. Écrire une suite d'instructions qui inverse l'ordre de ces trois valeurs situées à partir de la tête de la pile.

```
pop eax         ; eax = val1 (sommet)
```

```

pop ebx      ; ebx = val2
pop ecx      ; ecx = val3

push eax     ; empile val1 (maintenant au fond)
push ebx     ; empile val2
push ecx     ; empile val3 (au sommet)

```

En convention du C :

Inverser sans passer par des registres :

fonction_inverser:

```

    push ebp
    mov ebp, esp

    ; Les 3 valeurs sont déjà sur la pile
    ; On les dépile et les rempile pour les inverser

    pop ecx   ; récupère val1 (addr retour)
    pop ebx   ; récupère val2
    pop ecx   ; récupère val3

    push ecx   ; remets val3 (au sommet)
    push ebx   ; remets val2
    push eax   ; remets val1 (au fond)

    ; Ici la pile est inversée

    pop ebp
    ret

```

Appel depuis main :

```

push 10      ; val1
push 20      ; val2
push 30      ; val3
call fonction_inverser
; Après : pile inversée [30, 20, 10]

```

7. (RPS) Écrire une suite d'instructions qui place en tête de pile le maximum entre les valeurs des registres eax et ebx.

```

; eax et ebx contiennent deux valeurs
; On veut mettre le maximum sur la pile

cmp eax, ebx ; compare eax et ebx
jge eax_max  ; si eax >= ebx, saute à eax_max

; sinon ebx est plus grand
push ebx
jmp fin

```

```
eax_max:
push eax
```

```
fin:
; Le maximum est maintenant au sommet de la pile
```

8. (RSB) On suppose que `eax` et `ebx` contiennent des valeurs positives. Écrire une suite d'instructions qui remplace le contenu de `ebx` par la valeur initiale de `ebx` plus la valeur initiale de `eax`. L'utilisation des instructions `add` et `sub` n'est pas autorisée. L'utilisation des instructions `inc` et `dec` est recommandée.

```
; eax et ebx contiennent des valeurs positives
; On veut : ebx = ebx + eax (sans add/sub)
```

```
cmp eax, 0      ; compare eax avec 0
je fin          ; si eax = 0, on a fini
```

```
boucle:
inc ebx        ; ebx++
dec eax        ; eax--
cmp eax, 0     ; est-ce que eax est arrivé à 0 ?
jne boucle     ; si non, on continue
```

```
fin:
; ebx contient maintenant ebx_initial + eax_initial
```

9. (RPSB) Écrire une suite d'instructions qui dépile toutes les valeurs nulles situées à partir de la tête de la pile.

```
; On dépile tant qu'on trouve des 0 au sommet de la pile
```

```
boucle:
pop eax        ; dépile la valeur au sommet dans eax
cmp eax, 0     ; est-ce que c'est 0 ?
jne fin        ; si ce n'est pas 0, on arrête
jmp boucle     ; sinon, on continue (dépile le prochain)
```

```
fin:
; Tous les 0 ont été dépilés
; eax contient la première valeur non-nulle
; (ou une valeur quelconque si la pile était vide)
```

10. (RPSB) On suppose que `eax` contient une valeur entière et que la pile contient un nombre suffisant de valeurs. Écrire une suite d'instructions qui dépile les valeurs de la pile qui sont strictement supérieures à la valeur contenue dans `eax`. Le processus s'arrête dès qu'une valeur inférieure à celle de `eax` figure en tête de pile.

```
; eax contient une valeur de référence
```

; On dépile les valeurs strictement supérieures à eax

boucle:

```
pop ebx      ; dépile une valeur dans ebx
cmp ebx, eax  ; compare ebx avec eax
jle fin      ; si ebx <= eax, on arrête
jmp boucle    ; sinon, on continue (ebx > eax)
```

fin:

; Tous les éléments strictement supérieurs à eax ont été dépilés
; ebx contient la première valeur <= eax (ou une valeur quelconque)

11. (RPSB) On suppose que la pile contient un nombre suffisant de valeurs et que la tête de pile contient une valeur non nulle. Écrire une suite d'instructions qui place dans eax la plus grande valeur de la pile rencontrée jusqu'à ce qu'une valeur nulle soit trouvée.

; La pile contient des valeurs, on cherche le maximum jusqu'à trouver 0

pop eax ; dépile la première valeur (non-nulle selon l'énoncé)

boucle:

```
pop ebx      ; dépile la valeur suivante
cmp ebx, 0    ; est-ce 0 ?
je fin       ; si oui, on arrête

cmp ebx, eax  ; compare ebx avec le max actuel (eax)
jle boucle    ; si ebx <= eax, on saute (pas nouveau max)

mov eax, ebx  ; sinon, ebx est plus grand -> eax = ebx
jmp boucle    ; on continue
```

fin:

; eax contient le maximum rencontré avant le 0

12. (RPSF) Écrire une fonction (respectant les conventions du C) qui prend trois arguments a, b et c et qui renvoie 1 si a = b = c et 0 sinon.

; Fonction qui vérifie si a = b = c
; Arguments : a à [ebp+8], b à [ebp+12], c à [ebp+16]
; Retour : 1 si égaux, 0 sinon (dans eax)

fonction_egal:

```
push ebp
mov ebp, esp
push ebx
push ecx
```

```

; Récupère les 3 arguments
mov eax, [ebp+8] ; eax = a
mov ebx, [ebp+12] ; ebx = b
mov ecx, [ebp+16] ; ecx = c

; Vérifie si a = b
cmp eax, ebx
jne pas_egal ; si a != b, retour 0

; Vérifie si b = c
cmp ebx, ecx
jne pas_egal ; si b != c, retour 0

; Si on arrive ici : a = b et b = c
mov eax, 1 ; retour 1
jmp fin_fonction

pas_egal:
mov eax, 0 ; retour 0

fin_fonction:
pop ecx
pop ebx
pop ebp
ret

[ebp+16] = argument 3 (c)
[ebp+12] = argument 2 (b)
[ebp+8] = argument 1 (a)
[ebp+4] = adresse de retour
[ebp+0] = ancien ebp

```

13. (RPSBF) On suppose que l'on dispose d'une fonction pow (écrite selon les conventions du C) qui prend en premier argument une valeur x (entière signée) et en second argument une valeur n (entière positive). Cette fonction renvoie la valeur $x^{>n</sup>}$. On ne se soucie pas des dépassement de capacité éventuels ici. Écrire une suite d'instructions qui empile successivement $n^1, n^2, n^3, \dots, n^k$ où n est la valeur de eax et k est la valeur de ebx.

```

; eax = n (la base)
; ebx = k (le nombre de puissances à calculer)
; On doit empiler  $n^1, n^2, n^3, \dots, n^k$ 

mov ecx, 1 ; ecx = exposant courant (commence à 1)

boucle:
cmp ecx, ebx ; compare l'exposant avec k
jg fin ; si exposant > k, on arrête

```

```

; Prépare l'appel à pow(n, exposant)
push ecx      ; 2e argument : l'exposant
push eax      ; 1er argument : la base (n)
call pow      ; appel à pow
add esp, 8    ; nettoie les 2 arguments

; eax contient maintenant n^exposant
push eax      ; empile le résultat

inc ecx       ; exposant++
jmp boucle

```

fin:

; La pile contient $[n^1, n^2, \dots, n^k]$

14. (RPSBFC) Écrire une fonction (respectant les conventions du C) qui prend comme argument l'adresse d'une chaîne de caractères et qui renvoie sa longueur (et donc le nombre de caractères situés avant le caractère nul 0).

```

; Fonction qui calcule la longueur d'une chaîne
; Argument : adresse de la chaîne à [ebp+8]
; Retour : longueur dans eax

```

fonction_longueur:

```

push ebp
mov ebp, esp
push ebx

mov ebx, [ebp+8] ; ebx = adresse de la chaîne
mov eax, 0       ; eax = compteur de caractères

```

boucle:

```

cmp byte [ebx], 0 ; regarde le caractère courant
je fin           ; si c'est 0 (fin de chaîne), on arrête

inc eax          ; incrémente le compteur
inc ebx          ; passe au caractère suivant
jmp boucle

```

fin:

```

pop ebx
pop ebp
ret

```

15. (RPSBFC) Écrire une fonction (respectant les conventions du C) qui prend comme arguments les adresses de deux chaînes de caractères et qui renvoie 1 si les deux chaînes de caractères sont égales et 0 sinon.

```
; int strequal(char *s1, char *s2);
```

```
strequal:
```

```
    push ebp  
    mov  ebp, esp  
    push ebx  
    push ecx
```

```
    mov  ebx, [ebp+8] ; ebx = s1  
    mov  ecx, [ebp+12] ; ecx = s2
```

```
boucle:
```

```
    mov  al, [ebx] ; al = *s1  
    mov  dl, [ecx] ; dl = *s2  
    cmp  al, dl  
    jne  pas_egal ; caractères différents -> 0
```

```
    cmp  al, 0  
    je   egal ; même caractère et c'est 0 -> fin des deux chaînes -> 1
```

```
    inc  ebx ; s1++  
    inc  ecx ; s2++  
    jmp  boucle
```

```
pas_egal:
```

```
    mov  eax, 0  
    jmp  fin
```

```
egal:
```

```
    mov  eax, 1
```

```
fin:
```

```
    pop  ecx  
    pop  ebx  
    pop  ebp  
    ret
```

16. (RPSBFT) Écrire une fonction (respectant les conventions du C) qui prend comme arguments les adresses de deux tableaux et une taille n. Les deux tableaux sont formés de n cases de 4 octets. La fonction renvoie 1 si les deux tableaux sont égaux et 0 sinon.

```
tableaux_egaux:
```

```
    push ebp  
    mov  ebp, esp
```

```
    mov  ebx, [ebp+8] ; ebx = t1
```



```

mov ecx, [ebp+12] ; ecx = t2
mov edx, [ebp+16] ; edx = n

mov eax, 1 ; résultat = 1 (égaux) par défaut

```

```

boucle:
    cmp edx, 0
    je fin ; si n == 0 → fin, restent égaux

```

```

    mov eax, [ebx] ; eax = t1[i]
    cmp eax, [ecx] ; compare t1[i] et t2[i]
    jne pas_egal ; si différents → 0

```

```

    add ebx, 4 ; t1++
    add ecx, 4 ; t2++
    dec edx ; n--
    jmp boucle

```

```

pas_egal:
    mov eax, 0 ; pas égaux

```

```

fin:
    pop ebp
    ret

```

17. (RPSBFM) Écrire une fonction (respectant les conventions du C) qui prend comme arguments une adresse adr, un entier positif n et un caractère c. Cette fonction écrit aux adresses allant de adr à adr + n - 1 la valeur c.

```

; void fill(char *adr, int n, char c);

```

```

fill:
    push ebp
    mov ebp, esp

    mov ebx, [ebp+8] ; ebx = adr
    mov ecx, [ebp+12] ; ecx = n (compteur)
    mov al, [ebp+16] ; al = c (on n'utilise que 1 octet de eax)

```

```

boucle:
    cmp ecx, 0
    je fin ; si n == 0 → terminé

    mov [ebx], al ; *adr = c
    inc ebx ; adr++
    dec ecx ; n--
    jmp boucle

```

fin:

```
pop ebp
ret
```

```
.....
; ajouter : marque l'entier i comme premier (met 1 dans tab[i])
```

ajouter :

```
push ebp
mov ebp, esp
push ebx
```

```
mov ebx, [ebp + 8] ; ebx = i
mov byte [tab + ebx - 2], 1 ; tab[i-2] = 1 (car tab[0] = entier 2)
```

```
pop ebx
pop ebp
ret
```

```
.....
; lire : lit la valeur de tab[i] (retour dans eax)
```

lire :

```
push ebp
mov ebp, esp
push ebx
```

```
mov ebx, [ebp + 8] ; ebx = i
mov al, [tab + ebx - 2] ; al = tab[i-2]
movzx eax, al ; etend al dans eax
```

```
pop ebx
pop ebp
ret
```

```
.....
; initialiser : remplit tab de 1 (tous les nombres sont potentiellement premiers)
```

initialiser :

```
push ebp
mov ebp, esp
push ebx
push ecx
```

```
xor ebx, ebx ; ebx = 0
```

init_loop:

```
cmp ebx, N - 1
jge init_fin
```

```
mov byte [tab + ebx], 1 ; tab[ebx] = 1
inc ebx
```

init_fin:

```
pop ecx
pop ebx
pop ebp
ret
```

```

; premier_suivant : trouve le prochain entier marque comme premier
; Arguments : (1) i (à partir duquel chercher)
; Renvoi : le premier suivant dans eax
premier_suivant :

```

```
push ebp
mov ebp, esp
push ebx
```

```
mov ebx, [ebp + 8]    ; ebx = i
```

```
ps_loop:
```

```
cmp ebx, N
jge ps_fin_rien
```

```
cmp byte [tab + ebx - 2], 1
je ps_trouve
```

```
inc ebx
jmp ps_loop
```

ps_trouve:

```
mov eax, ebx
jmp ps_sortie
```

ps_fin_rien:

```
mov eax, 0 ; ou -1, selon convention
```

ps_sortie:

```
pop ebx
pop ebp
ret
```

```

.....
; effacer_multiples : marque tous les multiples de i comme non premiers
; Arguments : (1) i
effacer_multiples :

```

```
push ebp
mov ebp, esp
push ebx
```

```

    push ecx

    mov ebx, [ebp + 8]    ; ebx = i
    mov ecx, ebx
    add ecx, ebx          ; ecx = 2*i (premier multiple à marquer)

em_loop:
    cmp ecx, N
    jge em_fin

    mov byte [tab + ecx - 2], 0 ; marquer comme non premier
    add ecx, ebx            ; ecx += i (multiple suivant)
    jmp em_loop

em_fin:
    pop ecx
    pop ebx
    pop ebp
    ret

;-----
; main : implementer le crible d'Ératosthène
main :
    push ebp
    mov ebp, esp

    ; Initialiser le tableau
    call initialiser

    mov eax, 2            ; commencer à 2
crible_loop:
    cmp eax, N
    jge crible_fin

    ; lire tab[eax]
    push eax
    call lire
    add esp, 4

    ; si tab[eax] = 1, alors eax est premier
    cmp eax, 1
    jne crible_suivant

    ; marquer tous les multiples
    mov eax, [esp - 4]    ; recuperer i
    push eax
    call effacer_multiples
    add esp, 4

```

```
crible_suivant:  
    mov eax, [esp - 4]  
    inc eax  
    jmp crible_loop
```

```
crible_fin:  
    pop ebp  
    jmp end
```